DCLAB R&D · COMPREHENSIVE GAP ANALYSIS & IMPROVEMENT PLAYBOOK

What Is Still Missing in DCLab —and Exactly How to Fix It

> Setting aside Hierarchical Task Planning (`TP-01` to `TP-07`) as an ongoing track, this report explains every remaining gap across DCLab's product, agents, ML engine, models, benchmarks, and infrastructure—in plain, simple words, with concrete step-by-step fixes.

Date: 9 November 2026 · Checkout: `08c7bac` (`dclab-core 0.157.0`) · Scope: Entire repository excluding ongoing Hierarchical Task Planning

Contents

1. 1. The Big Picture in Plain Words
2. 2. All 18 Gaps at a Glance
3. 3. Category A: Home, Lead Agent & Routing Gaps
4. 4. Category B: ML Engine, Problem Kinds & Safety Gaps
5. 5. Category C: DCLab's Own Models & Training Data Gaps
6. 6. Category D: Product & Vision Gaps (Phases 14–19)
7. 7. Category E: Benchmarks, Sandbox & Open-Source Gaps
8. 8. Step-by-Step Improvement Order (What to Do First)

---

1. The Big Picture in Plain Words

Over Phases 1 to 13, DCLab built an extraordinary foundation: a 19-page studio, a portable solving library (`dclab_core`), 17 problem kinds, deterministic leakage checks, a single-read holdout lock, a 77-point decision ledger, unified agent memory, an 8-model training store, and a Claude-first Lead Agent.

If we set aside **Hierarchical Task Planning** (`TP-01A..TP-07`) as an already-active track, what is holding DCLab back today falls into **five simple truths**:

1. **The front door (Home & Lead Agent) is 100% safe, but talks too much when the user is already clear, costs ~2.3× too much, and misses the route ~20% of the time.**
2. **The ML engine runs 15 problem kinds, but 3 kinds fail their sealed end-to-end test (segmentation, uplift, regression), 14 kinds are still marked "experimental", and the leakage scanner misses multi-column math leaks when column names are hidden.**
3. **DCLab has collected ~100,000+ training records, but 0 of its 17 small decision "students" are trusted enough to act in the product yet, and the owner's ~8B Solver LLM has not been trained on a GPU yet.**
4. **Everything built for the long-term Vision (decision cards, live monitoring, retraining, company models) works in laboratory tests, but 0 real data teams use DCLab weekly via a simple `import dclab` SDK, and 0 real business outcomes have flowed back.**
5. **In DCLab Bench, 6 of 10 evaluation layers show red—partly from real gaps, and partly because old rejected experiments (like multi-agent debate) are still counted as active failures.**

An everyday analogy: Imagine DCLab is a self-driving car built in a test facility. The brakes (leakage & holdout protection) are world-class. The black-box recorder (experience log & training store) records every second. Now we need to: (1) stop the voice assistant from asking 3 questions when the driver says "Drive to work", (2) fix 3 gears that slip on steep hills (segmentation, uplift, regression), (3) graduate the student drivers from the simulator to real roads, and (4) hand the keys to the first 3–5 real drivers.

2. All 18 Gaps at a Glance

ID	Area	What Is Missing or Broken	Impact	Effort to Fix
A1	Lead Agent	Asks questions when the user's first sentence is already complete (`0/3` on clear requests).	CRITICAL	Small (1–2 days)
A2	Home Routing	Lead Agent misses route ~21–25% on sealed exams; chip vs. text conflicts need hardening (`R-01`).	CRITICAL	Small (2 days)
A3	Cost & Speed	Lead Agent costs ~2.3× raw Claude (€0.15/case), ~22s/turn, and €5 soft cap allowed a €1.08 runaway loop.	HIGH	Small (1–2 days)
B1	ML Engine	Sealed `system_tasks` gate failed (`26/30`): segmentation (`0/2`), uplift (`1/2`), regression (`1/2`).	CRITICAL	Medium (3–4 days)
B2	Leakage Audit	Heuristics miss multi-column sum leaks (`A + B = target`) when names are hidden, and flag `Time` as a leak.	HIGH	Small (2 days)
B3	Fraud / Entities	On real `Snap_fruad` , generic rules scored `0.019` PR-AUC vs `0.095` with pair/shared-key features.	HIGH	Medium (3–5 days)
B4	Engine Safety	Crash right after `final` scoring leaves `holdout_uses = 0` ; tabular splits lack a row-index hash.	HIGH	Small (2 days)
B5	Problem Kinds	`image` lacks local weights; `metric_change` is library-only; 7 kinds not in self-play; 13 kinds uncertified.	MEDIUM	Medium (1 week)
C1	Decision Students	17 students trained, only 2 passed sealed gate, and **0 serve above shadow mode** (overfit to synthetic tables).	HIGH	Medium (3–5 days)
C2	AutoML Model	Learned AutoML meta-model (`0.465` regret) loses to "always pick most frequent winner" (`0.395`).	MEDIUM	Small (2–3 days)
C3	Training Store	91% of `m1` data is prediction; `solution_design` in Gold is empty; only 5% of decisions have outcomes.	MEDIUM	Medium (3–4 days)
C4	Own 8B LLM	Exporters and LoRA recipes exist, but DCLab's own 8B Solver & Decision LLMs have 0 training runs yet.	MEDIUM	Needs GPU / M1 Max
D1	Python SDK & Users	No `import dclab` lightweight notebook SDK; 0 external data teams using DCLab weekly (Gaps G1, G2).	CRITICAL	Medium (1–2 weeks)
D2	Manager View	No dedicated Manager Decision Card page; promised gains are never checked against real outcomes (Gaps G4, G5).	HIGH	Medium (1 week)
D3	Live Outcome Loop	Scheduled batch scoring, late-outcome matching, and ~5% exploration share have never run live (Gaps G6–G8).	HIGH	Large (2–3 weeks)
D4	Company Optimizer	No cross-project optimizer combining many models (churn + uplift + risk) under one company budget (Gaps G9–G10).	MEDIUM	Large (after D3)
E1	DCLab Bench	6 of 10 layers are `red` and 2 are `amber` ; retired experiments keep layers red; 3 eval test files skipped.	HIGH	Small (2–3 days)
E2	Open Source & Sandbox	4 `dclabai` handoff bugs (multiclass text, `.env` init, catalog overwrite, timeout) + no container kernel sandbox.	MEDIUM	Medium (3–5 days)

3. Category A: Home, Lead Agent & Routing Gaps

The Home page and the Phase 13 Claude-first Lead Agent (`dclab_rnd/agents/case_agent.py`) are the front door of DCLab. Across 28 benchmark runs (`CEX-001` to `CEX-032`), the Lead Agent eliminated all 927 safety failures that raw Claude made. However, three practical gaps remain.

Gap A1 · The "Clear-Request Trap": Asking Questions When the User Is Already Clear

CRITICAL

Where: `dclab_rnd/agents/case_agent.py`, `dclab_core/case.py`, `dclab_rnd/prompts/case_agent.md`

In plain words

When a user types a complete, crystal-clear request on turn 1—for example, *"Predict Churn on my attached telco.csv table before the monthly renewal date so our retention team can call the top 100 risky customers"*—the Lead Agent still asks 1 to 3 follow-up questions before offering options or acting. On both `sealed_v2` and `sealed_v3`, it scored **0% (0 of 3)** on the `clear_request` trap.

Why it happens

Phase 13 taught the agent a 4-stage habit (*Listen → Reflect → Explore → Propose*) with "about 3 exchanges as the normal minimum". Without a hard check in code, the model treats asking a question on turn 1 as polite conversation.

How to improve it

1. **Add an "Act-First" readiness check in `dclab_core/case.py`:** Write a pure function `is_clear_on_arrival(case_file)` that returns `True` when the user's message + attached data already specify: (a) what they want (goal/route), (b) who/what unit it is for, and (c) the table (or that no table is needed, such as a pure learning question).
2. **Enforce it in `dclab_rnd/agents/case_agent.py`:** When `is_clear_on_arrival(case_file)` is `True`, inject a runtime instruction into the turn context: *"The case file is already complete. Do NOT ask a clarifying question; immediately run the requested data tool or present the 2–3 actionable options."*
3. **Verify on practice cases first:** Run `python -m dclab_rnd.agent_eval.conversation_exam --split dev` (capped under €1) to confirm `clear_request` passes 100% without hurting understanding on vague requests.

Gap A2 · Route Misses (~21–25%) & Chip-vs-Text Conflicts (Package `R-01`) CRITICAL

Where: `dclab_rnd/draft/intent.py`, `dclab_rnd/draft/chat.py`, `dclab_rnd/agents/case_agent.py`, `dclab_rnd/agentic/web/src/views/home.html`

In plain words

DCLab has 6 help tracks on Home: **Model** (`prediction`), **Analyze** (`analysis`), **Prepare** (`preparation`), **Report** (`reporting`), **Plan** (`business`), and **Learn** (`advice`). On `sealed_v2` and `sealed_v3`, the Lead Agent picked the wrong route on ~21% to 25% of conversations. Two things go wrong:

- When a user wants a prediction or analysis but hasn't uploaded their CSV yet, the agent often misroutes them to `advice` instead of keeping the route as `prediction` / `analysis` and asking for the file.
- When a user clicks a Home chip (e.g., **Learn**) but types a conflicting sentence (e.g., *"build a churn model on Telco"*), or refreshes the page mid-chat, the precedence between the clicked chip and the typed words is not fully audited (Package `R-01`).

How to improve it

1. **Separate "Missing Data" from "Advice":** In `dclab_core/case.py` and `case_agent.py`, if the user's goal is a prediction, analysis, cleaning, or report on their own business, keep that route even when `tables == 0`, and mark `missing_table` as the blocker to ask for—never downgrade the route to `advice`.
2. **Run the Haiku route verifier on every proposal (Fix S3):** In `case_agent.py`, `chip_checked` currently uses a fast Haiku check only when a chip was clicked. Run that same check whenever the agent proposes options or sets a route.
3. **Complete Package `R-01` (`engineering/planning/R_01.txt`):** Lock down the route precedence table in code and tests (`tests/test_home_intent.py`, `tests/test_case_agent.py`):
   - Explicit chip selection (`You picked`) wins unless the user explicitly changes their mind.
   - If the user's sentence contradicts their chip, surface the contradiction **once** clearly (*"You picked Learn, but asked to build a churn model—would you like to build the model or read how churn models work?"*).
   - Ensure a clicked chip survives browser refresh and never bypasses build/holdout approval guards.

Gap A3 · Lead Agent Cost (~2.3× Raw Claude), Turn Latency (~22s), and the €5 Soft-Cap Bug HIGH

Where: `dclab_rnd/settings.py`, `dclab_rnd/agents/case_agent.py`, `dclab_rnd/models/anthropic_client.py`

In plain words

In the benchmark post-mortem (`LEAD_AGENT_BENCHMARK_CONCLUSION.md`), the Lead Agent cost an average of **€0.147 per person** vs **€0.064** for Claude alone. Worse, in one conversation (`s3-02`), the agent hit its 9,800-token thinking/output limit 6 times in a row, spending **€1.08 and 12.5 minutes** on a single user because the default soft cap (`DCLAB_CASE_CAP_EUR`) was set to €5.00.

How to improve it

1. **Lower the default soft cap to €0.40:** Change `DCLAB_CASE_CAP_EUR` from `5.0` to `0.40` in `dclab_rnd/settings.py`, and add a hard guard in `dclab_rnd/agents/runtime.py` that aborts a loop if 2 consecutive calls hit `max_output_tokens`.
2. **Raise prompt-cache reuse from 46% to ~75%+:** Keep the system prompt, tools, and static evidence at the top of the message list, and update the `CaseFile` inside the same turn as the reply instead of making a separate uncached model call.
3. **Use adaptive thinking effort by turn complexity:** Use high thinking effort on turn 1 (understanding the problem and reading the data card), and standard effort on simple follow-up confirmations.

4. Category B: ML Engine, Problem Kinds & Safety Gaps

Once a problem is understood, DCLab's 5-stage engine (`dclab_rnd/studio/engine.py` and `dclab_rnd/kinds/`) builds and evaluates the model. Here are the five concrete gaps in the solving engine and how to close them.

Gap B1 · End-to-End Sealed Gate Failures on `segmentation`, `uplift`, and `regression` CRITICAL

Where: `dclab_rnd/agent_eval/system_tasks.py` , `dclab_rnd/kinds/segmentation.py` , `dclab_rnd/kinds/uplift.py` , `dclab_rnd/kinds/regression.py`

In plain words

DCLab built an end-to-end test called `system_tasks` that runs all 15 supported problem kinds from raw replica data through all 5 stages with zero human help and checks every decision. On the two unseen sealed seeds (`731` and `732` , commit `23558b6`), DCLab passed **26 of 30 tasks (86.7%)**, but failed the overall gate because:

- **Segmentation failed on both seeds (0 of 2)**;
- **Uplift failed on one seed (1 of 2)**;
- **Regression failed on one seed (1 of 2)**.

How to improve it

1. **Diagnose on dev seeds (never tune on sealed seeds 731/732):** Run `python -m dclab_rnd.agent_eval.system_tasks --split dev` and generate seeds `5..20` in a scratch check to see which acceptance assertion fails on segmentation, uplift, and regression.
2. **Fix the brittle kind rules:**
   - In `dclab_rnd/kinds/segmentation.py` (and `dclab_core/clustering.py`): check why the cluster count `k` or stability/scaling check fails when a seed has tighter cluster separation, and ensure identifier/unscaled trap columns are dropped before k-means stability is scored.
   - In `dclab_rnd/kinds/uplift.py` (and `dclab_core/uplift.py`): verify the treatment-arm balance check and post-treatment leak filter across seeds.
   - In `dclab_rnd/kinds/regression.py` : check the target-skew / log-transform / outlier-cap rule when synthetic seeds generate heavy-tailed targets.
3. **Verify on 20 dev seeds, then run one fresh sealed seed pair:** Only after 20/20 dev seeds pass for all 15 kinds, pin a new sealed seed pair (`system_tasks_v2`) and run it once.

Gap B2 · Leakage Scanner Misses Multi-Column Sum Leaks & Falsely Flags Time Columns HIGH

Where: `dclab_core/analyse.py`, `dclab_rnd/studio/leakage_review.py`, `dclab_rnd/tools.py`

In plain words

DCLab checks one column at a time for suspicious names or abnormally high single-column correlation with the target. Because of this:

- If two columns add up to the target (like `casual_riders + registered_riders = total_rentals` in bike sharing) and their names are hidden (`f001`, `f002`), neither column looks like a 1.0 leak alone, so the scanner misses them.
- On `online_shoppers`, 8 of 9 post-visit page-activity columns had moderate single-column correlation and normal-looking names, so blind rules missed them.
- On `credit_card_fraud`, the rules falsely flagged the `Time` column as a leak instead of using it as the time-split column.

How to improve it

1. **Add a 2-column & row-sum identity detector in `dclab_core/analyse.py`:** On training rows only, check whether any pair of numeric columns satisfies `A + B = Y`, `A - B = Y`, or `A × B = Y` (within `1e-6` tolerance), or whether a subset of numeric columns sums to `Y`. Flag matching columns as `exact_combination_leak` even when names are obfuscated.
2. **Add a 2-feature tree probe:** Fit a shallow depth-2 decision tree on pairs of suspect high-leverage columns on training folds to catch multi-column proxy leaks.
3. **Exempt monotonic timestamp / elapsed-time columns from leakage exclusion:** In `leakage_review.py`, if a column is identified as the table's primary time/order column (`column.time`), route it to `split.strategy = time` rather than `declared_leakage_features`.

Gap B3 · Rare-Event / Fraud Feature Gap on Multi-Entity Problems (`Snap_fruad`) HIGH

Where: `dclab_core/behaviour.py`, `dclab_core/entities.py`, `dclab_core/programme.py`

In plain words

On the owner's real delivery-fraud problem (`Snap_fruad`: 134,362 rides, 100 confirmed fraud cases), DCLab's automatic rules scored **0.019 PR-AUC**, whereas the owner's custom script scored **0.095 PR-AUC** (5× higher). Why? Because collusion fraud lives in **relationships between entities**—such as the same biker and customer riding together 8 times in one day, or 5 accounts sharing the same phone/device hash—whereas DCLab's current `entity_behaviour` recipe only computes single-entity rolling averages.

How to improve it

1. **Add pair-history & shared-key velocity features to `dclab_core/behaviour.py`:** When a table has two or more entity/ID columns (e.g., `customer_id`, `biker_id`, `device_id`), automatically compute strictly past-only (before ride timestamp `t`, inside training folds):
   - `pair_count_prior` and `pair_share_of_customer_rides` for `(E1, E2)` pairs;
   - `distinct_counterparties_24h` (how many distinct bikers/devices this customer used recently);
   - `time_since_last_event` for each entity and pair.
2. **Re-test on the `delivery_fraud` replica and `Snap_fruad`:** Verify that fold-safe pair features close the gap from `0.019` toward `0.095` without any future leakage.

Gap B4 · Crash Window During Final Holdout Scoring & Missing Tabular Split Digest HIGH

Where: `dclab_rnd/studio/engine.py`, `dclab_rnd/studio/programme.py`, `dclab_rnd/expansion/datasets.py`

In plain words

Found during the `TP-01A` architecture audit (`TASK_PLANNING_ARCHITECTURE.md` Section 6):

- **The crash gap:** In `engine._run_stage` and `studio.programme.run`, the code computes the holdout score first and saves the incremented `holdout_uses` counter to disk/PostgreSQL afterward. If the server crashes or is killed mid-write after computing the score, the saved project still says `holdout_uses = 0`, allowing an accidental second holdout read.
- **The split fingerprint gap:** While specialized kind machines record a SHA-256 fingerprint of the exact train/holdout row indices, standard tabular binary/multiclass/regression projects only record the file hash, random seed, and split ratio.

How to improve it

1. **Write a durable "holdout-intent receipt" before scoring:** In `engine._run_stage` (when `stage == "final"`) and `studio.programme.run`, persist a durable `holdout_claim` record (incrementing `holdout_uses` or setting `holdout_state = "in_flight"`) to the project store **before** touching holdout rows. If a crash happens, recovery blocks automatic retry and asks a human to reconcile.
2. **Save `split_sha256` in `stage_data`:** In `dclab_rnd/expansion/datasets.py::prepare_bundle` and `studio/engine.py::stage_data`, compute and store the SHA-256 hash of the sorted training and holdout row indices.

Gap B5 · Uncertified Problem Kinds (13 of 15), 7 Kinds Missing from Self-Play, and Local Image Weights MEDIUM

Where: `dclab_rnd/certify.py`, `dclab_rnd/lake/selfplay.py`, `dclab_rnd/kinds/`

In plain words

DCLab defines 3 maturity levels for every problem kind (`ADDING_CAPABILITY.md`): `experimental` → `certified` → `product_default`. Today:

- Only **2 kinds** (`binary` and `matching`) are `certified`; the other 13 supported kinds and all 5 modifiers are still `experimental` because they haven't been run through `certify.py` on 2 real public datasets each.
- **7 kinds** (`metric_change`, `entity_tables`, `matching`, `ranking`, `segmentation`, `streams`, `anomaly`) are skipped by `selfplay.py` because their outcomes aren't a single plain column.
- `image` classification has decision rules in `dclab_core/image_decisions.py`, but awaits a small local vision backbone to run in the Studio engine.

How to improve it

1. **Certify the remaining kinds batch by batch:** Pair each of the 13 kinds with 2 existing public/OpenML datasets and run `python -m dclab_rnd.certify --kind <kind>` to verify all 5 checks (`detection`, `traps`, `baseline`, `artifact`, `llm_off`).
2. **Teach `selfplay.py` to read non-column outcomes:** Update `dclab_rnd/lake/selfplay.py` to read `kind_fields` (cutoff tables for `entity_tables`, pair labels for `matching`, query groups for `ranking`, unlabeled budgets for `anomaly` and `segmentation`) so all kinds produce training trajectories.

---

5. Category C: DCLab's Own Models & Training Data Gaps

DCLab's strategy (`THREE_MODELS.md`, `MODELS_INVENTORY.md`) is to train three families of models from its own captured data: (1) fast per-point **Decision Students**, (2) an ~8B **Solver LLM**, and (3) a **Workflow/Combination Model**. Here is what is missing today.

Gap C1 · 0 of 17 Decision "Students" Serve Above Shadow Mode HIGH

Where: `dclab_core/decision_models.py`, `dclab_rnd/real_homework.py`, `dclab_rnd/ledger/routing.py`

In plain words

DCLab trained 17 small scikit-learn models ("students") to make fast technical choices (like *"Is this column an ID?"*, *"Is this column a leak?"*, *"What is the task framing?"*). Today:

- **15 of 17 failed their sealed exam** (`decision_gate_v1`), and **0 of 17 are turned on above shadow mode**.
- Why? First, students trained only on invented tables learned artificial shortcuts and dropped sharply on real OpenML tables (e.g., `task.framing` dropped from 91.6% to 74.8%).
- Second, when DCLab retrained them with 650+ real OpenML tables (`models_real_v1/v2`), dev accuracy jumped to **99.6%** on `column.semantic_role` and **98.1%** on `task.framing`, but `DMG-014` missed the 80% macro-precision gate by just **0.15% (79.85%)** because of false `identifier` predictions on high-cardinality numbers.
- Third, promoting a student from `shadow` to `act` requires human blind labels (`Admin → Decisions`) on real disagreements.

How to improve it

1. **Fix the `identifier` precision dip in `models_real_v2`:** Add two clean structural features to `dclab_core/answerers.py::features()`: `is_float_with_decimals` (floats with fractional parts are almost never IDs, even when 100% unique) and `is_monotonic_step_1`.
2. **Run the sealed exam on real OpenML tables for the top 3 points:** Test `column.semantic_role`, `task.framing`, and `feature.leakage_suspect` on a fresh held-out OpenML partition.
3. **Label 30–50 blind disagreements on `Admin → Decisions`:** Use the existing Blind Labelling Desk (`docs/guides/BLIND_LABELLING.md`) so the two passing students can legally graduate from `shadow` to `flag` / `act_with_check`.

Gap C2 · Learned AutoML Meta-Model Loses to "Most Frequent Winner" MEDIUM

Where: `dclab_rnd/lake/automl.py`, `evidence/campaigns/automl_v1/`

In plain words

DCLab trained a meta-model to predict which algorithm family (ExtraTrees, LightGBM, XGBoost, Logistic Regression, etc.) will win on a table before running full cross-validation (`AML-002`). While it beat "always pick XGBoost" (regret `0.465` vs `0.786`), it lost to the simple rule "always pick the most frequent winner across datasets" (regret `0.395`).

How to improve it

1. **Add fast "landmarking" features:** Table row/column counts alone cannot tell whether tree splits or linear weights fit a target better. Add two 50-millisecond landmarkers on a 1,000-row training sample: (a) 3-fold decision-stump score vs. (b) 3-fold ridge/logistic score (`linear_vs_stump_gap`).
2. **Train on real OpenML screen results (`real_screen.py`)** instead of only synthetic self-play tables.

Gap C3 · Training Store (`m1–m8`) Imbalance & Empty `solution_design` Gold Set MEDIUM

Where: `dclab_rnd/store/coverage.py`, `dclab_rnd/lake/`, `dclab_rnd/prompts/`

In plain words

Running `make coverage` (`PHASE12_TRAINING_DATA_PLAN.md`) revealed four data gaps in the training store:

- **91% of `m1` examples come from the `prediction` track**; the other 5 Home tracks (`analysis`, `preparation`, `reporting`, `business`, `advice`) and `m8_retrieval` (only 9 rows) have very few examples.
- **The `solution_design` family in the Training Lake's Gold tier is empty (`0 rows`)**, even though every verified self-play run already has a `ProblemCard`, `DataCard`, and validated `SolutionWorkflow`.
- **Only ~5% of `m1` and `m3` decisions have linked outcomes** showing whether the decision led to a good holdout/business result.
- **6 older prompt files** in `dclab_rnd/prompts/` still ask the model to write its reasoning inside the JSON reply text (which Phase 13 banned in `37.14` in favor of `display: "summarized"` thinking blocks).

How to improve it

1. **Populate `solution_design` in Gold:** Extend `dclab_rnd/lake/selfplay.py` to emit one `solution_design` training example (`ProblemCard + DataCard → SolutionWorkflow`) for each of the 193+ verified self-play solves.
2. **Clean the 6 legacy prompts (`37.14`):** Remove inline "explain your reasoning" fields from the 6 legacy prompts listed in `test_thinking_capture.py` so every model purpose relies cleanly on native summarized thinking.
3. **Run `make outcomes-follow`** after project runs so holdout scores attach as measured outcomes to every upstream decision in the project.

Gap C4 · The Owner's Fine-Tuned LLMs (`Solver LLM` & `Decision LLM`) Have Not Been Trained Yet MEDIUM

Where: `research/llm-fine-tuning/experiments/sft/train_lora.py`, `dclab_rnd/models/tuned.py`

In plain words

All the plumbing to train and serve DCLab's own ~8B Qwen3-class models (`make lake`, `make solver-data`, `make sft-v3`, `make sft-dev`, 30% anti-forgetting replay sets, frozen exams, and `python -m dclab_rnd.models.tuned plan|shadow|promote|retire`) is 100% built and tested. However, no actual GPU LoRA training run has been executed yet (awaiting the owner's 64GB M1 Max or cloud GPU).

How to improve it

1. Once the 64GB M1 Max or a cloud GPU is ready, close Gap C3 first (so `solution_design` and verified code are in the mix).
2. Run `train_lora.py` on the decontaminated export (`python -m dclab_rnd.lake export --target solver`), score the frozen code and solver exams before and after, and deploy in **shadow mode first** via `python -m dclab_rnd.models.tuned shadow solver_answer`.

---

6. Category D: Product & Vision Gaps (Phases 14–19)

In `docs/vision/plan.html`, the owner identified 13 strategic gaps (`G1–G13`) between today's R&D studio and the full DCLab Vision. Putting aside `G3` (the Lead Agent, covered in Category A above), they form four clear product milestones.

Gap D1 · No Lightweight Python SDK (`import dclab`) & Zero External Weekly Teams (Gaps G1, G2) CRITICAL

Where: Phase 14 of `docs/vision/plan.html` · Builds on `dclab_core/` , `dclab_rnd/testers/`

In plain words

Data scientists live inside their own Jupyter, VS Code, or Colab notebooks. Today, using DCLab requires launching a separate web application (`make notebook`). A data scientist cannot simply write `import dclab` inside their existing notebook to check a DataFrame for leakage, run a leakage-safe 5-stage baseline, or get an honest holdout report. Because of that friction—and because outreach hasn't started—**zero external teams use DCLab weekly**.

How to improve it (Phase 14)

1. **Create a 4-function Python SDK facade in `dclab_core` (or a thin `dclab` wrapper):**
   - `dclab.audit(df, target="Churn", moment="before renewal")` → returns a clean table of leaky/ID columns and split advice.
   - `dclab.check_notebook("my_work.ipynb")` → runs the static copilot and prints mistakes with proof.
   - `dclab.solve(df, target="Churn", moment="...")` → runs the 5-stage engine locally and returns the model + honest holdout interval.
   - `dclab.brief(result)` → renders the 1-page Markdown/HTML Decision Brief.
2. **Onboard 3–5 friendly data teams for 4 weeks:** Give them the SDK + `make tester-setup` (`docs/guides/TESTERS.md`) so everything runs 100% on their laptop and only consented redacted records are shared via `make share` .

Gap D2 · No Dedicated Manager Page & Unverified Decision Promises (Gaps G4, G5) HIGH

Where: Phase 15 of `docs/vision/plan.html` · Builds on `dclab_core/decision.py` , `dclab_rnd/agentic/web/src/views/brief.html`

In plain words

A business manager does not want to read ROC-AUC, Brier scores, or fold standard deviations. They want a single **Decision Card** that says: *"Call these 250 customers this week. Cost: €1,250. Expected saved revenue: €8,400 to €11,200. Why: based on 1,400 held-out cases."* And 30 days later, they want to see: *"You promised €8,400–€11,200; actual observed result was €9,100."* Today, the Decision Brief page still mixes technical evaluation with business value, and promised gains are not tracked against later reality.

How to improve it (Phase 15)

1. **Add a clean "Manager View" toggle on the Decision Brief page (`brief.html`):** Show only the action list size, cost, expected net gain with 95% interval, interactive budget/cost sliders (`what_if` from `dclab_core/decision.py`), and plain-language risks.
2. **Log the "Promise Record":** When a manager signs off on a Decision Card, write a `decision.promised` record with the expected gain interval and entity keys so `ledger/follow.py` can score the promise when outcomes arrive.

Gap D3 · The Live Loop Has Never Run in a Company: Batch Scoring, Late Outcomes & 5% Exploration (Gaps G6, G7, G8) HIGH

Where: Phase 16 of `docs/vision/plan.html` · Builds on `dclab_core/serving.py`, `dclab_core/live.py`, `dclab_core/retrain.py`

In plain words

DCLab can export a model (`studio/code.py`, `studio/service.py`), log predictions (`serving.py`), check drift (`live.py`), and compare champion vs. challenger models (`retrain.py`). However:

- There is no one-command scheduled batch runner (e.g., score new rows every Monday morning from a warehouse table or folder).
- Real outcomes arrive weeks later (e.g., whether a customer actually churned 30 days after the call) and need automatic key+window joining.
- Without a **~5% exploration share** (acting differently on a small random slice of cases), the company only observes outcomes for the cases the model picked, blinding future retraining and uplift models.

How to improve it (Phase 16)

1. **Add an `explore_share` parameter (default `0.05` when agreed) to `dclab_core/decision.py`:** Stamp each scored row with `picked_by: "model" | "exploration"` and its selection propensity weight (`1 / p`) for unbiased future evaluation and retraining.
2. **Add a scheduled batch-scoring job in `dclab_rnd/jobs/`:** Let a project schedule weekly/daily batch scoring + outcome ingestion from a connected table (`dclab_rnd/connectors/`) and trigger `dclab_core/retrain.py` when enough new labelled outcomes accumulate.

Gap D4 · Combining Many Models into One Company Decision & Multi-Tenant Cloud (Gaps G9–G12) MEDIUM

Where: Phases 17–19 of `docs/vision/plan.html` · Builds on `dclab_core/programme.py`, `dclab_core/decision.py`, `deploy/`

In plain words

A real company eventually runs several models on the same customer: a Churn model says *"high risk of leaving"*, a Credit/Fraud model says *"medium risk of unpaid bill"*, and an Uplift model says *"discount offer will backfire"*. Today, each project decides in isolation. There is no cross-project **Company Decision Model** that takes all predictions for customer `C_104`, applies company-wide budget/contact rules, resolves conflicts, and chooses one optimal action.

How to improve it (Phases 17–19, after Phase 16)

1. **Build `dclab_core/company_decision.py` on a planted Telecom replica first:** Create a replica with 3 models (churn risk, customer lifetime value, offer uplift) + a shared call-centre capacity + a veto rule ("never send a discount if fraud risk > 10%"), and use constrained linear optimization to maximize net value.
2. **Apply to one real company after Phase 16's outcome loop is running**, then package company-specific models (Phase 18) and multi-tenant cloud hosting (Phase 19, `deploy/`).

7. Category E: Benchmarks, Sandbox & Open-Source (`dclabai`) Gaps

Gap E1 · 6 of 10 Layers in `DCLab Bench` Show Red (Partly Because Rejected Experiments Are Counted as Active) HIGH

Where: `dclab_rnd/bench/__init__.py` , `docs/guides/BENCHMARK.md`

In plain words

When you run `make bench-read` , 6 of the 10 layers show `red` and 2 show `amber` . Some of those red lights are real bugs we listed above (like `system_tasks` `26/30` or the Lead Agent `clear_request` trap). **However, several red lights are not product bugs at all:** DCLab tested research hypotheses (such as multi-agent `debate` in `ORC-005` , Optuna `search_v2` , or uncalibrated v1 students), proved they failed their gate, and rightly kept them **switched off** in the product. Because their campaign JSON files still sit in `evidence/campaigns/` , `bench` marks the whole layer `red` forever.

How to improve it

1. **Separate "Active Product Path" from "Retired/Off Research Arms" in `dclab_rnd/bench/` :** Add a `RETIRED_TRACKS` or `status: "experimental_off"` classification in `dclab_rnd/bench/` so `make bench-read` reports two clear columns: **Active Product State** (what serves users today) vs. **Research Experiments Archive** (hypotheses tested and kept off).
2. **Add formal gate functions for `retrieval_v1` and `memory_value_v2` :** Both already passed their numbers (`0.872` recall and `0/30` repeated mistakes), so adding their gate verdict turns the `retrieval and memory` layer from `amber` to `green` .
3. **Run `make bench LAYERS=software` so the `software` layer records its `product-e2e` pass and turns from `empty` to `green` .

Gap E2 · Four Open-Source `dclabai` Handoff Leftovers & Notebook Kernel Sandbox MEDIUM

Where: `docs/guides/OPEN_SOURCE_HANDOFF.md` , `dclab_rnd/studio/` , `scripts/export_open.py`

In plain words

Two engineering hygiene items remain from the open-source handoff and MLE-bench study:

1. **Four small bugs listed in `OPEN_SOURCE_HANDOFF.md` :**
   - TF-IDF text features currently run on binary classification only, not multiclass;
   - When a user runs `dclab init` without a model and later adds `ANTHROPIC_API_KEY` to `.env` , the doctor/status check doesn't auto-pick it up without re-init;
   - `make check-all` in a git worktree can touch `training_records/STORE_CATALOG.json` ;
   - `make product-e2e` has a tight 40-second timeout on slower machines.
2. **Notebook Kernel Sandbox (`33.7` in `MLE_BENCH_LESSONS_PLAN.md`):** While the Studio notebook kernel only receives training rows and zero API keys, user/agent code cells run in a local Python subprocess without an AST/file-open/container sandbox (`strace` / `openat` guard or Docker isolation), which is needed before shared cloud hosting.

How to improve it

1. Fix the 4 small `OPEN_SOURCE_HANDOFF.md` items in a single cleanup package and re-run `scripts/export_open.py --check` .
2. Implement Package `33.7` (`dclab_core/code_check.py` AST guard blocking network/process/holdout-file reads + optional container kernel runner in `dclab_rnd/studio/`).

8. Step-by-Step Improvement Order (What to Do First)

You do not need to fix all 18 gaps at once. Here is the most effective order of work—starting with the highest-impact, lowest-cost fixes that immediately improve DCLab's intelligence and turn its benchmarks green, followed by the real-user

rollout.

Step 1: Fix Front Door (A1–A3) → **Step 2: Fix Engine & Safety (B1, B2, B4)** →
Step 3: Clean Bench & Real Students (E1, C1) → **Step 4: Build SDK & First Teams (D1, D2)** →
Step 5: Live Loop & Own Models (D3, C4, D4)

Priority Step	Packages / Fixes Included	What Changes When Done	Estimated Time
Step 1 · Make Home & Lead Agent Sharp and Fast	**Gaps A1, A2, A3** (Act-first check on clear requests, `R-01` Home chip routing audit, Haiku route check on proposals, €0.40 soft cap, 75% prompt caching)	Home stops asking unnecessary questions on clear requests, keeps the right route when no file is attached yet, respects chip selections, and cuts per-case Claude cost in half.	3–4 days
Step 2 · Turn the ML Engine 30/30 Green & Crash-Safe	**Gaps B1, B2, B4** (Fix `segmentation`, `uplift`, and `regression` on dev seeds; add 2-column sum leak detector; persist holdout-intent receipt & tabular split SHA-256)	`system_tasks` passes all 15 kinds end-to-end; obfuscated sum leaks are caught; a crash during `final` can never cause a second holdout read.	4–5 days
Step 3 · Graduate the First Real-Table Students & Clean DCLab Bench	**Gaps E1, C1, C3** (Separate retired research arms in `bench`; fix false `identifier` precision on `models_real_v2`; label 30 blind decisions; export `solution_design` to Gold)	`DCLab Bench` accurately reflects active product health; the first 2–3 decision students (`semantic_role`, `leakage_suspect`, `task.framing`) graduate above shadow mode; Training Lake Gold has full workflow designs.	4–5 days
Step 4 · Ship the 4-Call Python SDK, Manager Card & First 3–5 Real Teams	**Gaps D1, D2, B3, E2** (Phase 14 `import dclab` SDK, Phase 15 Manager Decision Card view, pair-interaction fraud features, 4 `dclabai` handoff fixes)	Data scientists can use DCLab in 3 lines inside their own notebooks; managers get a 1-page action/gain card; 3–5 real teams start using DCLab weekly on their own data.	2 weeks
Step 5 · Run the Live Outcome Loop & Train the ~8B Solver LLM	**Gaps D3, C4, D4, B5** (Phase 16 batch scoring + late outcomes + 5% exploration share; GPU LoRA run for the 8B Solver LLM; Phase 17 multi-model optimizer)	Real business outcomes flow back into DCLab; promised gains are checked against reality; DCLab's own 8B model serves in shadow; multiple models combine into one company decision.	Ongoing (Phases 16–19)

Summary takeaway: None of these gaps require redesigning DCLab's architecture. The foundation (`dclab_core` + `dclab_rnd`) is already built. Steps 1, 2, and 3 are small, targeted code and rule improvements that can be completed right now in this repository, while Steps 4 and 5 take DCLab into real companies to fulfill the 1 November 2026 Vision.